COMP0222 Simultaneous Localisation and Mapping

Lecture 03: Properties and Scalability of Kalman Filter Linear SLAM

Simon Julier

s.julier@ucl.ac.uk

Sajad Saeedi

s.saeedi@ucl.ac.uk

Overview

- In the last lecture, we went through and derived the Kalman Filter-based SLAM algorithm for all-linear systems
- In this lecture, we'll look at the convergence properties of a SLAM system
- We'll also look at how the solution scales with map size
- We'll also look at what approaches can be used to improve the scalability

Structure

- Long-term Convergence Behaviour
- Computational and Storage Costs
- The Critical Role of Cross Correlations
- Ways to Improve Scalability

Long-term Convergence Behaviour

- **Long-term Convergence Behaviour**
- ***Computational and Storage Costs***
- ***The Critical Role of Cross Correlations***
- ***Ways to Improve Scalability***

Theoretical Properties of SLAM Algorithms

1. The *determinant* of *any* submatrix of the map covariance matrix *decreases monotonically* as observations are successively made

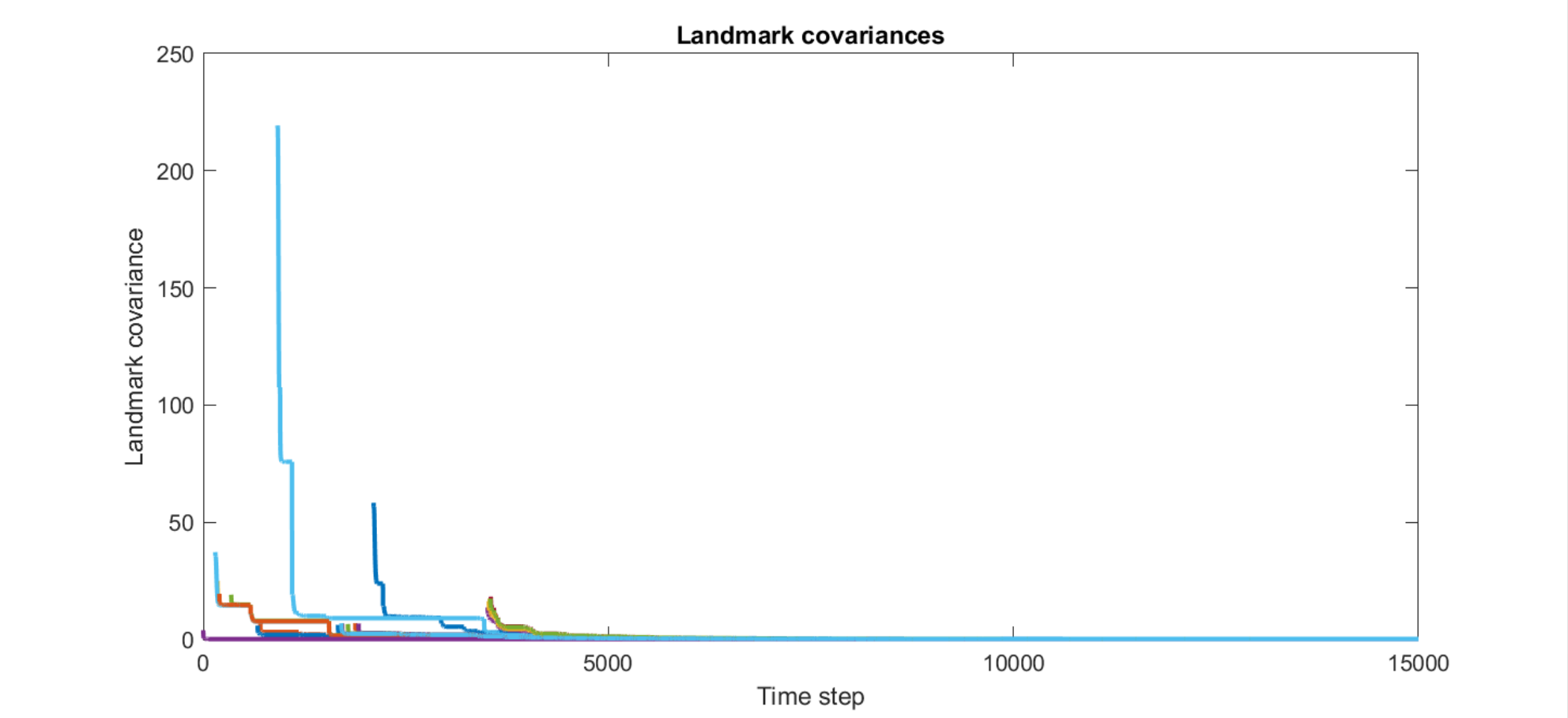
Full SLAM System Covariance Matrix

$$\mathbf{P}^{s}_{k|k} = \begin{bmatrix} \mathbf{P}^{\mathbf{xx}}_{k|k} & \mathbf{P}^{\mathbf{x}1}_{k|k} & \mathbf{P}^{\mathbf{x}2}_{k|k} & \mathbf{P}^{\mathbf{x}3}_{k|k} & \cdots & \mathbf{P}^{\mathbf{x}N_k}_{k|k} \\ \mathbf{P}^{1\mathbf{x}}_{k|k} & \mathbf{P}^{11}_{k|k} & \mathbf{P}^{12}_{k|k} & \mathbf{P}^{13}_{k|k} & \cdots & \mathbf{P}^{1N_k}_{k|k} \\ \mathbf{P}^{2\mathbf{x}}_{k|k} & \mathbf{P}^{21}_{k|k} & \mathbf{P}^{22}_{k|k} & \mathbf{P}^{23}_{k|k} & \cdots & \mathbf{P}^{2N_k}_{k|k} \\ \mathbf{P}^{3\mathbf{x}}_{k|k} & \mathbf{P}^{31}_{k|k} & \mathbf{P}^{32}_{k|k} & \mathbf{P}^{33}_{k|k} & \cdots & \mathbf{P}^{3N_k}_{k|k} \\ \vdots & \vdots & \ddots & \vdots & & \\ \mathbf{P}^{N_k\mathbf{x}}_{k|k} & \mathbf{P}^{N_k1}_{k|k} & \mathbf{P}^{N_k2}_{k|k} & \mathbf{P}^{N_k3}_{k|k} & \cdots & \mathbf{P}^{N_kN_k}_{k|k} \end{bmatrix}$$

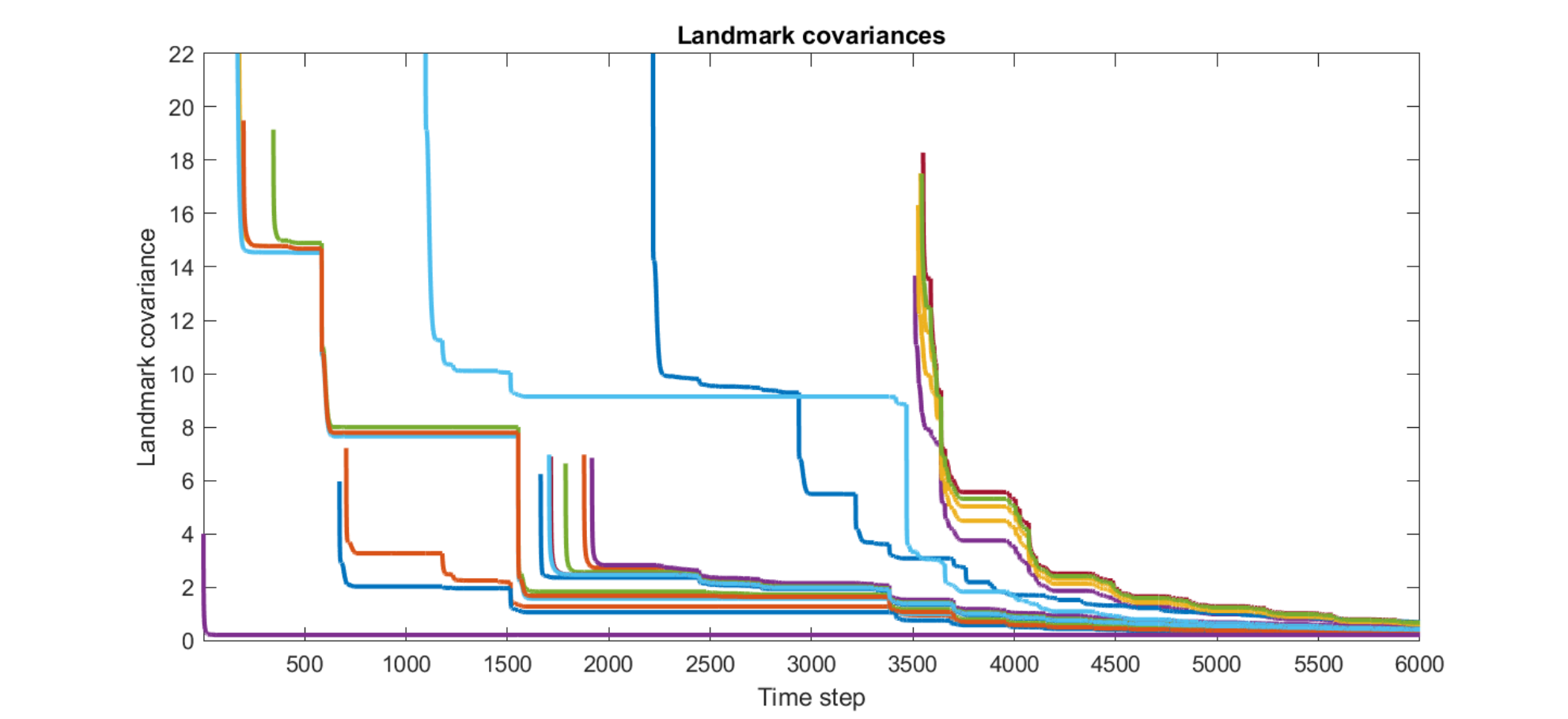
Map Covariance Block

$$\begin{bmatrix} \boxed{\mathbf{P}_{\mathbf{m}^1\mathbf{m}^1}} & \mathbf{P}_{\mathbf{m}^1\mathbf{m}^2} & \cdots & \mathbf{P}_{\mathbf{m}^1\mathbf{m}^{N_k}} \\ \mathbf{P}_{\mathbf{m}^2\mathbf{m}^1} & \boxed{\mathbf{P}_{\mathbf{m}^2\mathbf{m}^2}} & \cdots & \mathbf{P}_{\mathbf{m}^2\mathbf{m}^{N_k}} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{P}_{\mathbf{m}^{N_k}\mathbf{m}^1} & \mathbf{P}_{\mathbf{m}^{N_k}\mathbf{m}^2} & \cdots & \boxed{\mathbf{P}_{\mathbf{m}^{N_k}\mathbf{m}^{N_k}}} \end{bmatrix}$$

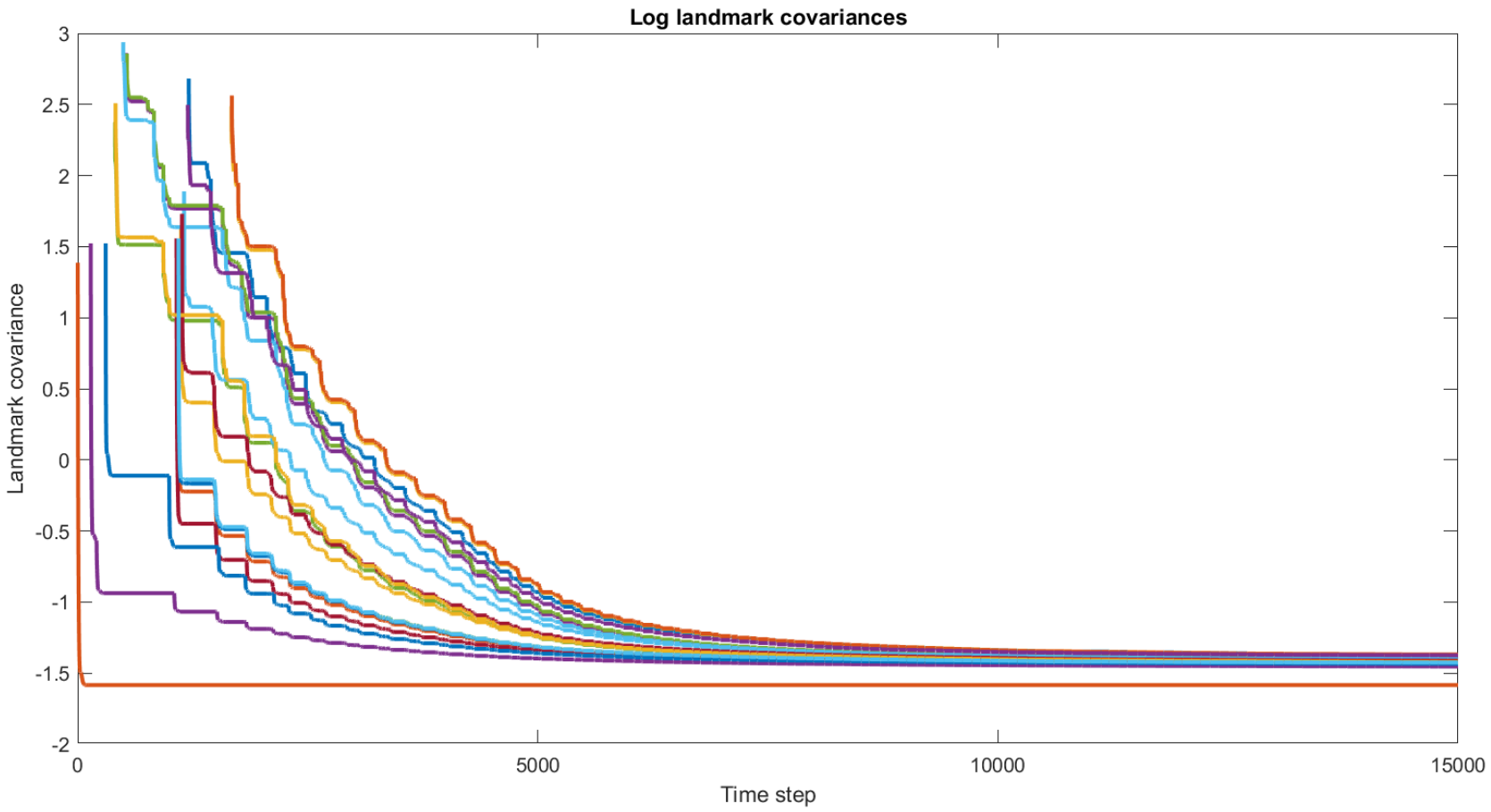
Landmark Covariance Behaviour



Landmark Covariance Behaviour (Zoomed)



Landmark Covariance Behaviour



Map Covariance Block

$$\begin{bmatrix} \mathbf{P}_{\mathbf{m}^1\mathbf{m}^1} & \mathbf{P}_{\mathbf{m}^1\mathbf{m}^2} & \cdots & \mathbf{P}_{\mathbf{m}^1\mathbf{m}^{N_k}} \\ \mathbf{P}_{\mathbf{m}^2\mathbf{m}^1} & \mathbf{P}_{\mathbf{m}^2\mathbf{m}^2} & \cdots & \mathbf{P}_{\mathbf{m}^2\mathbf{m}^{N_k}} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{P}_{\mathbf{m}^{N_k}\mathbf{m}^1} & \mathbf{P}_{\mathbf{m}^{N_k}\mathbf{m}^2} & \cdots & \mathbf{P}_{\mathbf{m}^{N_k}\mathbf{m}^{N_k}} \end{bmatrix}$$

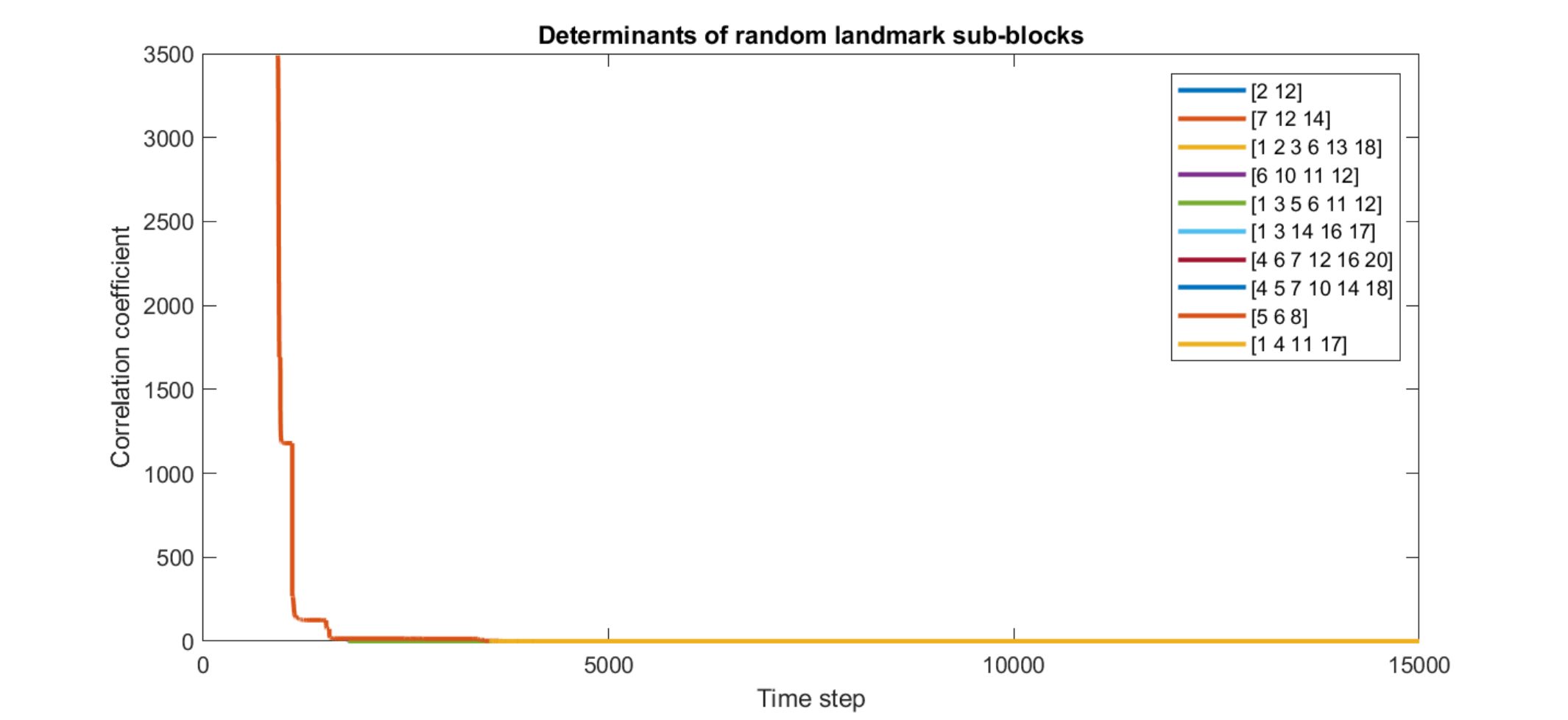
Map Covariance Block

$$\begin{bmatrix} \mathbf{P}_{\mathbf{m}^1\mathbf{m}^1} & \mathbf{P}_{\mathbf{m}^1\mathbf{m}^2} & \cdots & \mathbf{P}_{\mathbf{m}^1\mathbf{m}^{N_k}} \\ \mathbf{P}_{\mathbf{m}^2\mathbf{m}^1} & \mathbf{P}_{\mathbf{m}^2\mathbf{m}^2} & \cdots & \mathbf{P}_{\mathbf{m}^2\mathbf{m}^{N_k}} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{P}_{\mathbf{m}^{N_k}\mathbf{m}^1} & \mathbf{P}_{\mathbf{m}^{N_k}\mathbf{m}^2} & \cdots & \mathbf{P}_{\mathbf{m}^{N_k}\mathbf{m}^{N_k}} \end{bmatrix}$$

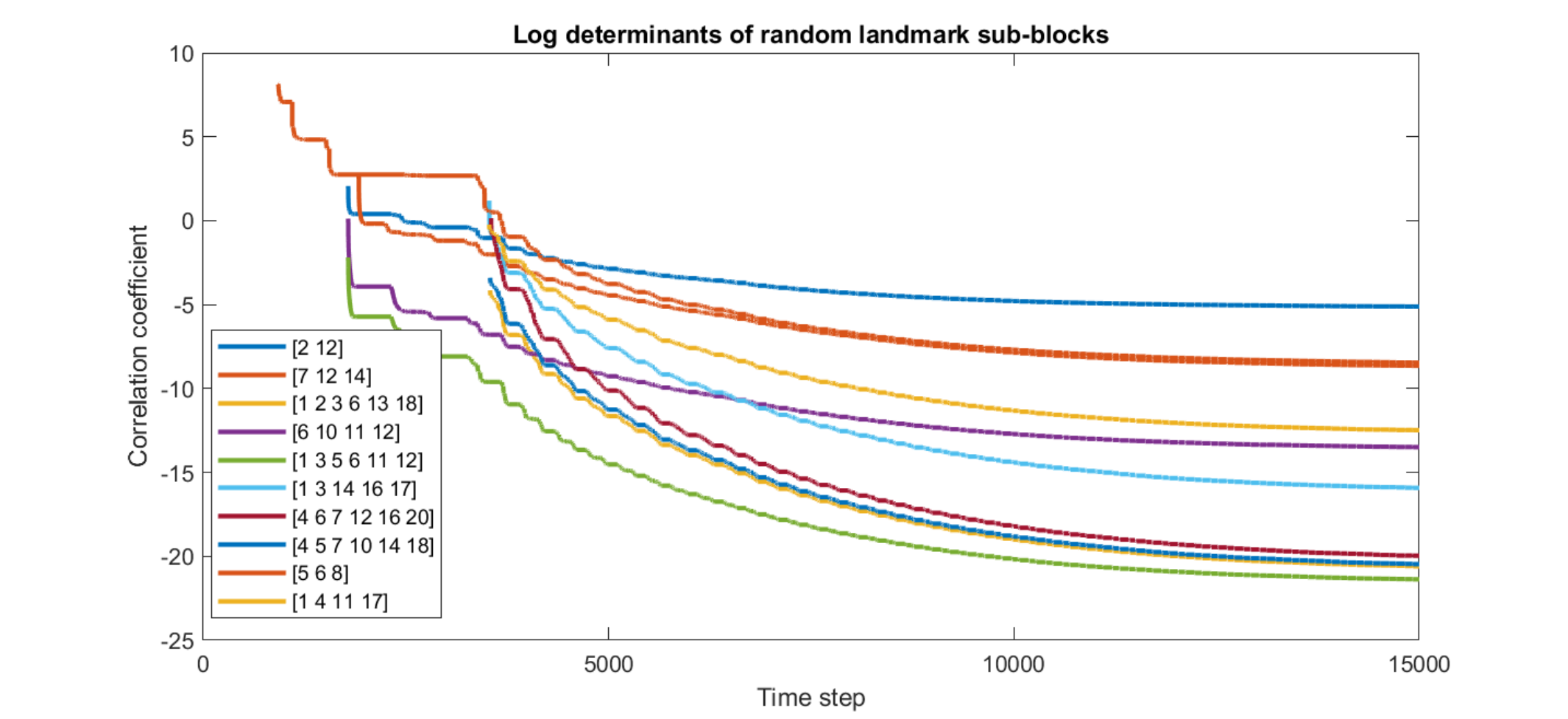
Map Covariance Block

$$\begin{bmatrix} \mathbf{P}_{\mathbf{m}^1\mathbf{m}^1} & \mathbf{P}_{\mathbf{m}^1\mathbf{m}^2} & \cdots & \mathbf{P}_{\mathbf{m}^1\mathbf{m}^{N_k}} \\ \mathbf{P}_{\mathbf{m}^2\mathbf{m}^1} & \boxed{\mathbf{P}_{\mathbf{m}^2\mathbf{m}^2}} & \cdots & \boxed{\mathbf{P}_{\mathbf{m}^2\mathbf{m}^{N_k}}} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{P}_{\mathbf{m}^{N_k}\mathbf{m}^1} & \boxed{\mathbf{P}_{\mathbf{m}^{N_k}\mathbf{m}^2}} & \cdots & \boxed{\mathbf{P}_{\mathbf{m}^{N_k}\mathbf{m}^{N_k}}} \end{bmatrix}$$

Determinants of Covariance Blocks



Log of Determinants of Covariance Blocks



Why Does This Happen?

- Because the landmarks don’t move and don’t suffer from process noise, their uncertainty to go up over time
- The Kalman filter update guarantees that the covariance can’t go up
- Therefore, the covariance can only stay the same or go down

Theoretical Properties of SLAM Algorithms

1. The *determinant* of *any* submatrix of the map covariance matrix *decreases monotonically* as observations are successively made
2. In the limit as the number of observations goes to infinity, the landmark estimates *become fully correlated*

Full Correlation of the Map Block

$$\begin{bmatrix} \mathbf{P}_{\mathbf{m}^1\mathbf{m}^1} & \mathbf{P}_{\mathbf{m}^1\mathbf{m}^2} & \cdots & \mathbf{P}_{\mathbf{m}^1\mathbf{m}^{N_k}} \\ \mathbf{P}_{\mathbf{m}^2\mathbf{m}^1} & \mathbf{P}_{\mathbf{m}^2\mathbf{m}^2} & \cdots & \mathbf{P}_{\mathbf{m}^2\mathbf{m}^{N_k}} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{P}_{\mathbf{m}^{N_k}\mathbf{m}^1} & \mathbf{P}_{\mathbf{m}^{N_k}\mathbf{m}^2} & \cdots & \mathbf{P}_{\mathbf{m}^{N_k}\mathbf{m}^{N_k}} \end{bmatrix}$$

Full Correlation of the Map Block

$$\begin{bmatrix} \mathbf{P}_{\#\#} & \mathbf{P}_{\#\#} & \cdots & \mathbf{P}_{\#\#} \\ \mathbf{P}_{\#\#} & \mathbf{P}_{\#\#} & \cdots & \mathbf{P}_{\#\#} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{P}_{\#\#} & \mathbf{P}_{\#\#} & \cdots & \mathbf{P}_{\#\#} \end{bmatrix}$$

Demonstrating the Convergence

- We constructed a scenario with 3 landmarks
- These landmarks are observed all the time
- The filter was run for different numbers of timesteps
- The final covariance matrix was output

Covariance Matrices (2k, 4k, 16k steps)

0.2551	0.0108	0.0907	0.0907	0.0907
0.0108	0.0015	-0.0000	-0.0000	0.0000
0.0907	-0.0000	0.0921	0.0901	0.0901
0.0907	-0.0000	0.0901	0.0921	0.0901
0.0907	0.0000	0.0901	0.0901	0.0921

0.2551	0.0108	0.0907	0.0907	0.0907
0.0108	0.0015	-0.0000	-0.0000	0.0000
0.0907	-0.0000	0.0914	0.0904	0.0904
0.0907	-0.0000	0.0904	0.0914	0.0904
0.0907	0.0000	0.0904	0.0904	0.0914

0.2551	0.0108	0.0907	0.0907	0.0907
0.0108	0.0015	-0.0000	0.0000	-0.0000
0.0907	-0.0000	0.0909	0.0906	0.0906
0.0907	0.0000	0.0906	0.0909	0.0906
0.0907	-0.0000	0.0906	0.0906	0.0909

Determinant of the Submatrix

- **A fully correlated covariance matrix has the property that its rank is equal to the dimension of a *single* landmark**
- **The determinant becomes 0**

```
P =

    0.0909    0.0909    0.0909
    0.0909    0.0909    0.0909
    0.0909    0.0909    0.0909
```

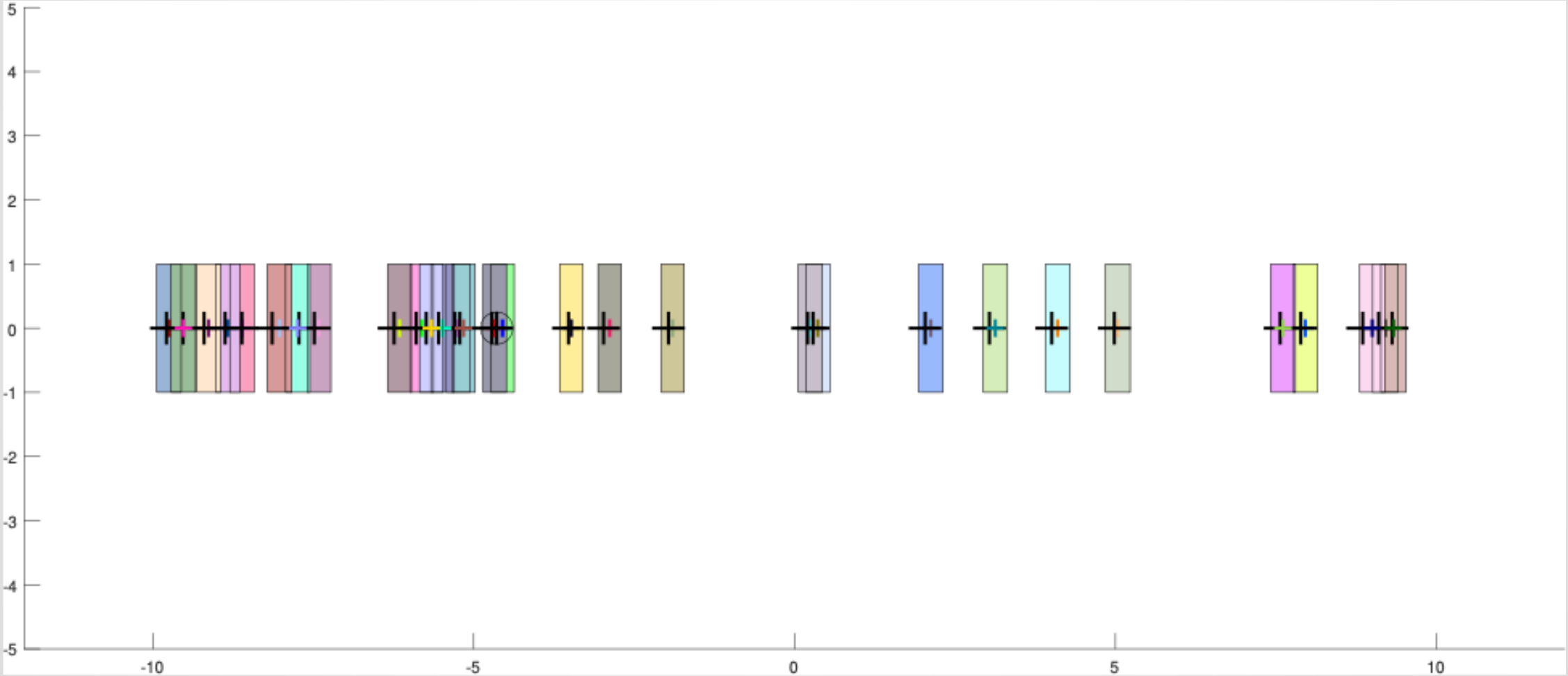
```
>> eig(P)

ans =

   -0.0000
    0.0000
    0.2727
```

Full Correlation of the Map Block

- **This means two things:**
  - **The covariance of the landmark estimates *becomes the same***
  - **The error in the landmark estimates all *become the same***
  - **The *relative error* between landmarks *goes to 0***

- **We can see this in a converged map for beadSLAM**

Full Correlation of the Map Block

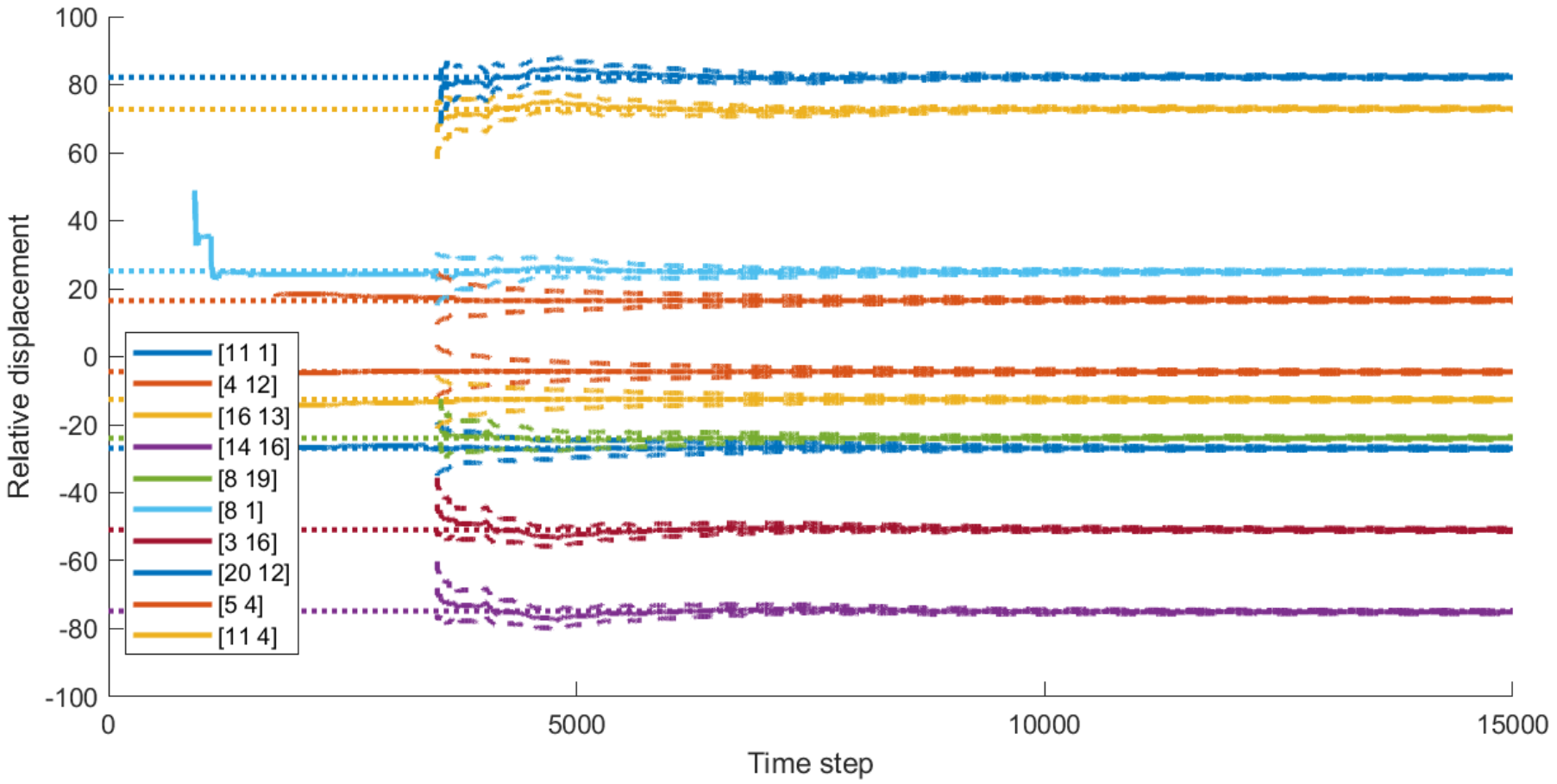


Relative Displacement Between Pairs of Landmarks

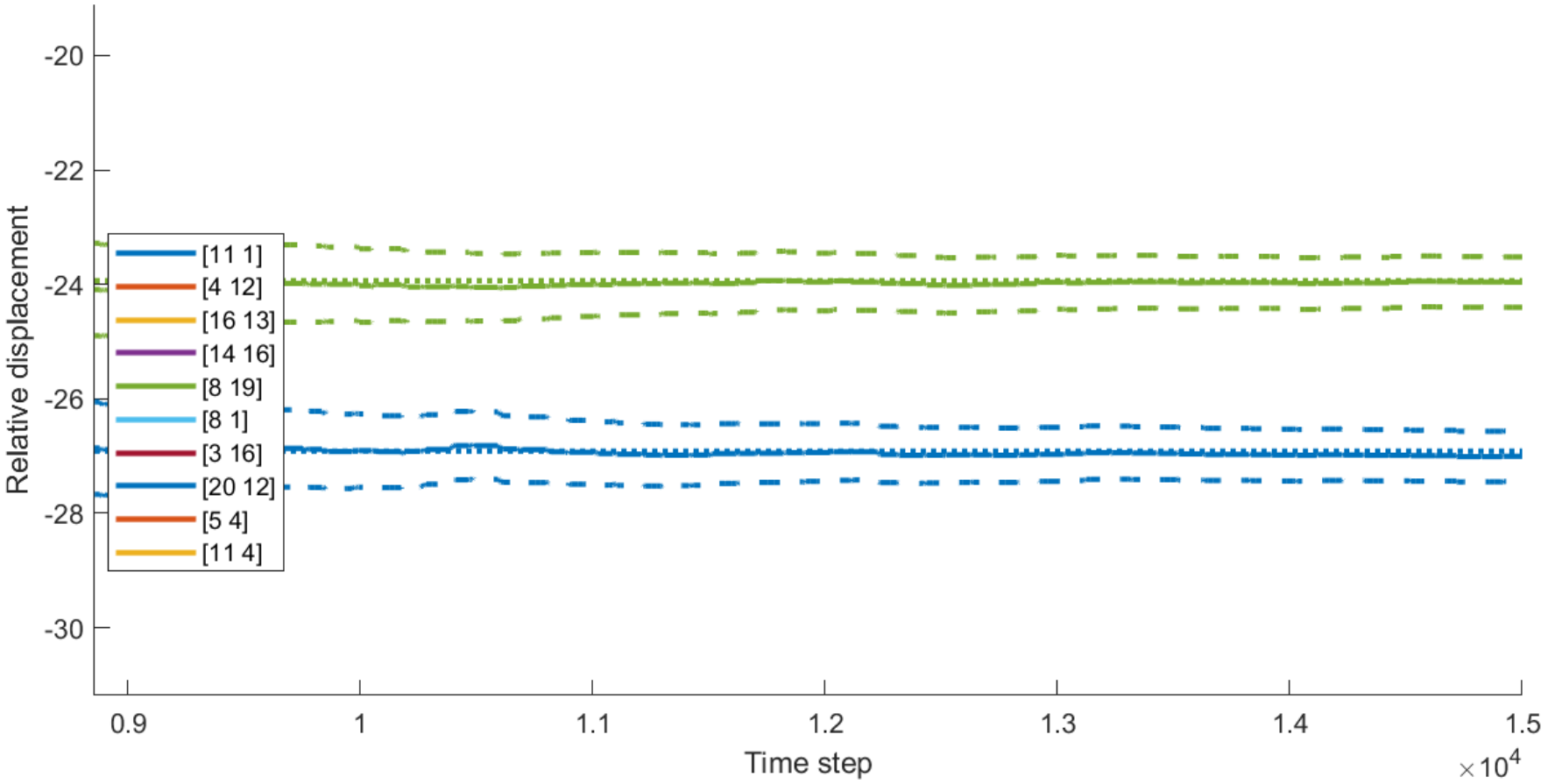
- **The relative displacement is the difference between the estimate of *two* landmarks,**

$$\delta \hat{m}^{ab}_{k|k} = \hat{m}^{a}_{k|k} - \hat{m}^{b}_{k|k}$$

Relative Displacement Between Pairs of Landmarks



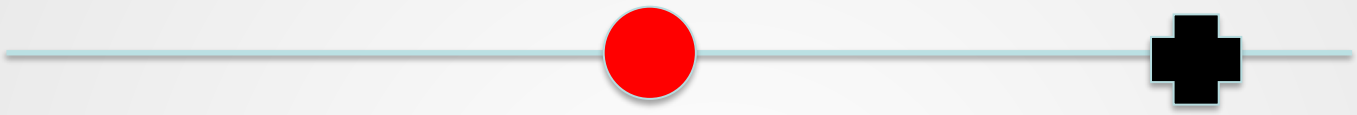
Relative Error Between Pairs of Landmarks (Zoomed)



Theoretical Properties of SLAM Algorithms

1. The *determinant* of *any* submatrix of the map covariance matrix *decreases monotonically* as observations are successively made
2. In the limit as the number of observations goes to infinity, the landmark estimates *become fully correlated*
3. In the limit, the covariance associated with any single landmark location estimate *is determined only by the initial covariance in the vehicle location estimate*

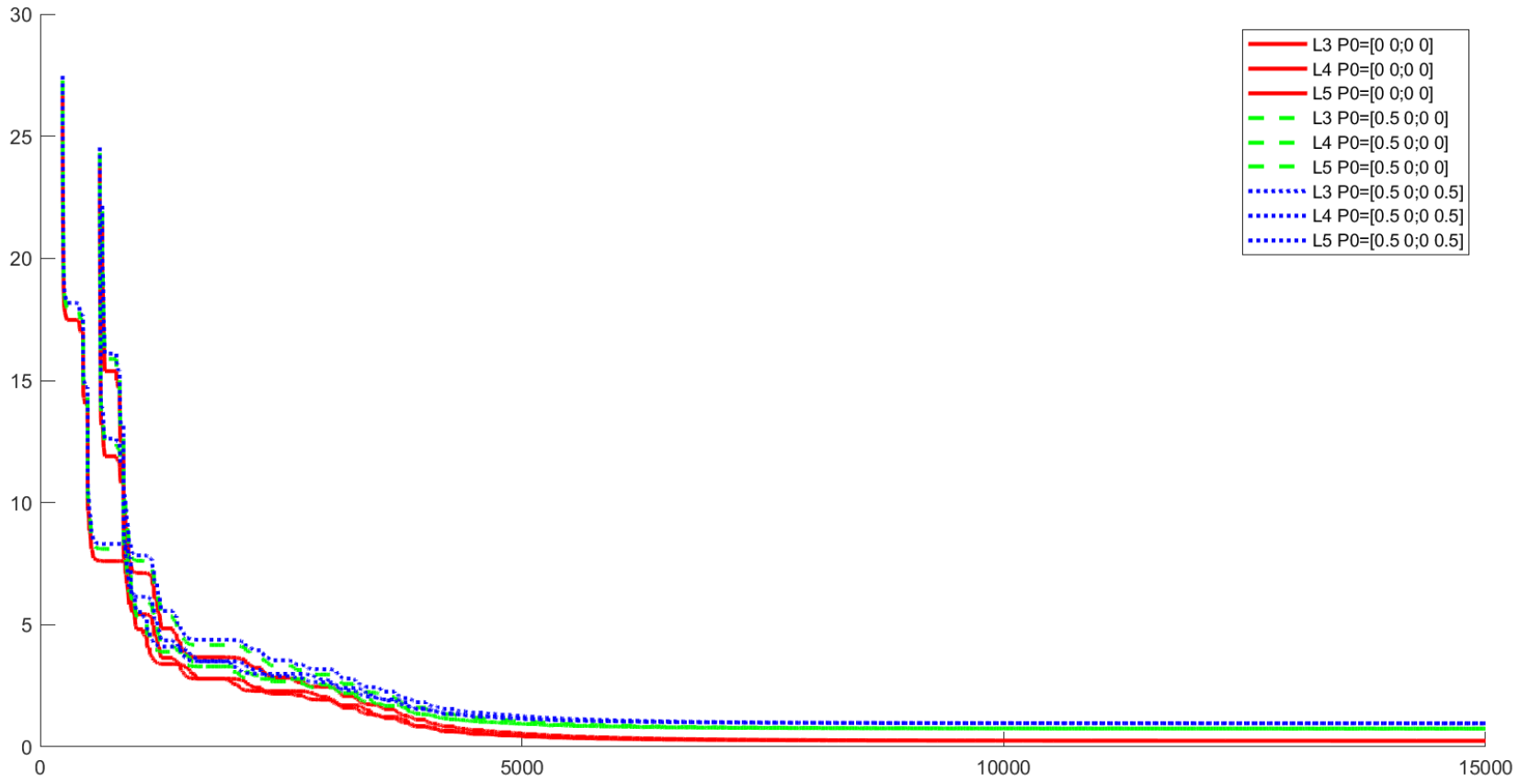
Limit Determined by the Initial Covariance



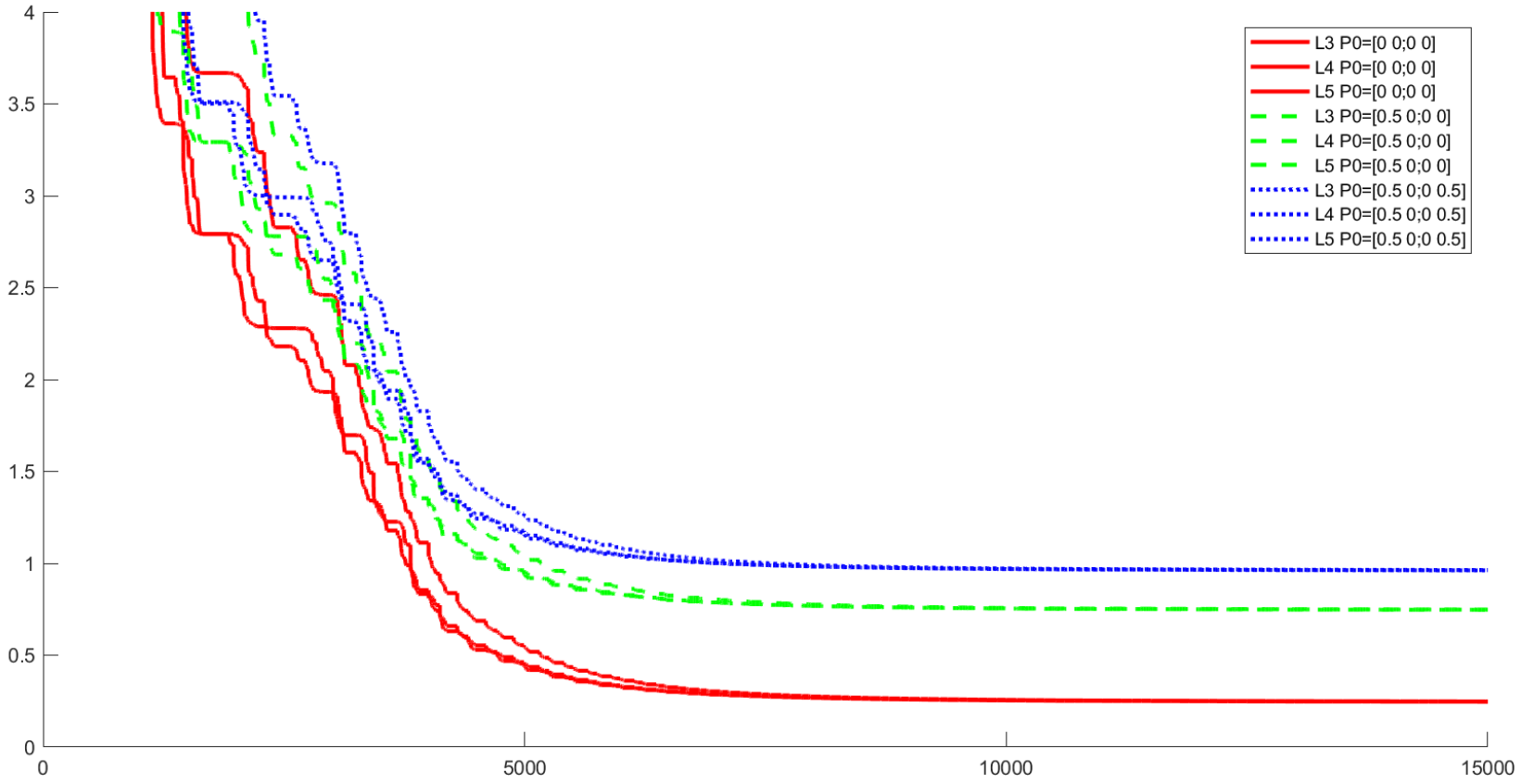
Demonstrating the Effects of Initial Covariance

- We ran the same beadSLAM instance using the same random number generator
- The only thing we changed was the initial covariance of the platform when the first landmark was observed

The Effects of Initial Covariance



The Effects of Initial Covariance (Zoomed)



Computational and Storage Costs

- *Long-term Convergence Behaviour*
- Computational and Storage Costs
- *The Critical Role of Cross Correlations*
- *Ways to Improve Scalability*

Scalability of Kalman Filter-Based SLAM

- When thinking about scalability, we need to consider two main issues:
  - Storage costs
  - Computational costs

Storage Costs of the State Vector

- **Recall that the state vector grows over time as we add more landmarks,**

$$\hat{\mathbf{s}}_{k|k} = \begin{pmatrix} \hat{\mathbf{x}}_{k|k} \\ \hat{\mathbf{m}}^{1}_{k|k} \\ \vdots \\ \hat{\mathbf{m}}^{N_k}_{k|k} \end{pmatrix} \qquad \mathrm{numel}\left(\hat{\mathbf{s}}_{k|k}\right) = 2 + N_k$$

Storage Costs of the Covariance Matrix

$$\mathbf{P}^{s}_{k|k} = \begin{bmatrix} \mathbf{P}^{\mathbf{xx}}_{k|k} & \mathbf{P}^{\mathbf{x}1}_{k|k} & \mathbf{P}^{\mathbf{x}2}_{k|k} & \mathbf{P}^{\mathbf{x}3}_{k|k} & \cdots & \mathbf{P}^{\mathbf{x}N_k}_{k|k} \\ \mathbf{P}^{1\mathbf{x}}_{k|k} & \mathbf{P}^{11}_{k|k} & \mathbf{P}^{12}_{k|k} & \mathbf{P}^{13}_{k|k} & \cdots & \mathbf{P}^{1N_k}_{k|k} \\ \mathbf{P}^{2\mathbf{x}}_{k|k} & \mathbf{P}^{21}_{k|k} & \mathbf{P}^{22}_{k|k} & \mathbf{P}^{23}_{k|k} & \cdots & \mathbf{P}^{2N_k}_{k|k} \\ \mathbf{P}^{3\mathbf{x}}_{k|k} & \mathbf{P}^{31}_{k|k} & \mathbf{P}^{32}_{k|k} & \mathbf{P}^{33}_{k|k} & \cdots & \mathbf{P}^{3N_k}_{k|k} \\ \vdots & \vdots & \ddots & \vdots & & \\ \mathbf{P}^{N_k\mathbf{x}}_{k|k} & \mathbf{P}^{N_k1}_{k|k} & \mathbf{P}^{N_k2}_{k|k} & \mathbf{P}^{N_k3}_{k|k} & \cdots & \mathbf{P}^{N_kN_k}_{k|k} \end{bmatrix}$$

Storage and KF-SLAM

- Naively, we could store the full covariance matrix, so

$$\mathrm{numel}\left(\mathbf{P}^{s}_{k|k}\right) = (2 + N_k)^2$$

- However, because the covariance matrix is symmetric, we can reduce this to

$$\mathrm{numel}\left(\mathbf{P}^{s}_{k|k}\right) = \frac{(2 + N_k)(3 + N_k)}{2}$$

Storage and KF-SLAM

- Therefore, the storage costs are

$$O\left(N_k^2\right)$$

Cost of Predicting the Mean

- **Recall the predicted state is**

$$\hat{\mathbf{s}}_{k+1|k} = \begin{pmatrix} \mathbf{F}_{k+1}\hat{\mathbf{s}}_{k|k} + \mathbf{B}_{k+1}\mathbf{u}_{k+1} \\ \hat{\mathbf{m}}^{1}_{k|k} \\ \vdots \\ \hat{\mathbf{m}}^{N_k}_{k|k} \end{pmatrix}$$

- **The computational cost is *constant* irrespective of map size**

Cost of Predicting the Covariance

- Recall the KF prediction equations are

$$\hat{\mathbf{s}}_{k+1|k} = \mathbf{F}^{\mathbf{s}}_{k+1}\hat{\mathbf{s}}_{k|k} + \mathbf{B}^{\mathbf{s}}_{k+1}\mathbf{u}_{k+1}$$

$$\mathbf{P}^{s}_{k+1|k} = \mathbf{F}^{\mathbf{s}}_{k+1}\mathbf{P}^{s}_{k|k}\mathbf{F}^{\mathbf{s},\top}_{k+1} + \mathbf{Q}^{s}_{k+1}$$

- The dominant term here is predicting the covariance

Cost of Predicting the Covariance

$$\mathbf{P}^{s}_{k+1|k} = \begin{bmatrix} \mathbf{F}^{\mathbf{s}}_{k+1}\mathbf{P}^{\mathbf{xx}}_{k|k}\mathbf{F}^{\mathbf{s},\top}_{k+1} + \mathbf{Q}^{s}_{k+1} & \mathbf{F}^{\mathbf{s}}_{k+1}\mathbf{P}^{\mathbf{x}1}_{k|k} & \cdots & \mathbf{F}^{\mathbf{s}}_{k+1}\mathbf{P}^{\mathbf{x}N_k}_{k|k} \\ \mathbf{P}^{1\mathbf{x}}_{k|k}\mathbf{F}^{\mathbf{s},\top}_{k+1} & \mathbf{P}^{11}_{k|k} & \cdots & \mathbf{P}^{1N_k}_{k|k} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{P}^{N_k\mathbf{x}}_{k|k}\mathbf{F}^{\mathbf{s},\top}_{k+1} & \mathbf{P}^{N_k1}_{k|k} & \cdots & \mathbf{P}^{N_kN_k}_{k|k} \end{bmatrix}$$

Cost of Predicting the Covariance

- **Therefore, the prediction computational cost is *linear* in the number of landmarks,**

$$O\left(N_k\right)$$

Augmentation

- **Augmentation adds a new landmark to the map,**

$$\hat{\mathbf{s}}_{k+1|k+1} = \begin{pmatrix} \hat{\mathbf{x}}^{*}_{k+1|k+1} \\ \hat{\mathbf{m}}^{1,*}_{k+1|k+1} \\ \vdots \\ \hat{\mathbf{m}}^{N_k,*}_{k+1|k+1} \\ \hat{\mathbf{m}}^{N_{k+1},*}_{k+1|k+1} \end{pmatrix}$$

Cost of Augmenting the New State

- **Recall we augmented the state from**

$$\hat{\mathbf{s}}_{k+1|k+1} = \mathbf{J}_{k+1}\hat{\mathbf{s}}^{*}_{k+1|k+1} + \mathbf{K}_{k+1}\mathbf{z}^{n}_{k+1}$$

$$\mathbf{P}^{s}_{k+1|k+1} = \mathbf{J}_{k+1}\mathbf{P}^{s,*}_{k+1|k+1}\mathbf{J}^{\top}_{k+1} + \mathbf{K}_{k+1}\mathbf{R}^{n}_{k+1}\mathbf{K}^{\top}_{k+1}$$

Cost of Augmenting the Covariance

$$\mathbf{P}^{s}_{k+1|k+1} = \begin{bmatrix} \mathbf{P}^{\mathbf{xx}}_{k+1|k+1} & \mathbf{P}^{\mathbf{x}1}_{k+1|k+1} & \cdots & \mathbf{P}^{\mathbf{x}N_k}_{k+1|k+1} & \mathbf{P}^{\mathbf{x}N_k}_{k+1|k+1}\mathbf{J}^{\top} \\ \mathbf{P}^{1\mathbf{x}}_{k+1|k+1} & \mathbf{P}^{11}_{k+1|k+1} & \cdots & \mathbf{P}^{1N_k}_{k+1|k+1} & \mathbf{P}^{1N_k}_{k+1|k+1}\mathbf{J}^{\top} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ \mathbf{P}^{N_k\mathbf{x}}_{k+1|k+1} & \mathbf{P}^{N_k1}_{k+1|k+1} & \cdots & \mathbf{P}^{N_kN_k}_{k+1|k+1} & \mathbf{P}^{N_kN_k}_{k+1|k+1}\mathbf{J}^{\top} \\ \mathbf{J}\mathbf{P}^{\mathbf{x}1}_{k+1|k+1} & \mathbf{J}\mathbf{P}^{\mathbf{x}2}_{k+1|k+1} & \cdots & \mathbf{J}\mathbf{P}^{\mathbf{x}N_k}_{k+1|k+1} & \mathbf{J}\mathbf{P}^{\mathbf{xx}}_{k+1|k+1}\mathbf{J}^{\top} + \mathbf{K}\mathbf{R}^{n}_{k+1}\mathbf{K}^{\top} \end{bmatrix}$$

Cost of Augmenting the Covariance

- **Therefore, once again the prediction computational cost is *linear* in the number of landmarks,**

$$O\left(N_k\right)$$

Computational Costs of the Update

- The update is given by the Kalman filter equations

$$\hat{\mathbf{s}}^{*}_{k+1|k+1} = \hat{\mathbf{s}}_{k+1|k} + \mathbf{W}_{k+1|k}\boldsymbol{\nu}_{k+1|k}$$

$$\mathbf{P}^{s,*}_{k+1|k+1} = \mathbf{P}^{s}_{k+1|k} - \mathbf{W}_{k+1|k}\mathbf{S}_{k+1|k}\mathbf{W}^{\top}_{k+1|k}$$

- There are no clever tricks with computing the update which can simplify it

Computational Costs of the Update

- The full analysis is rather complicated, but when the platform updates with M_k landmarks, the computational cost is

$$O\left(N_k^2 M_k\right)$$

In Summary…

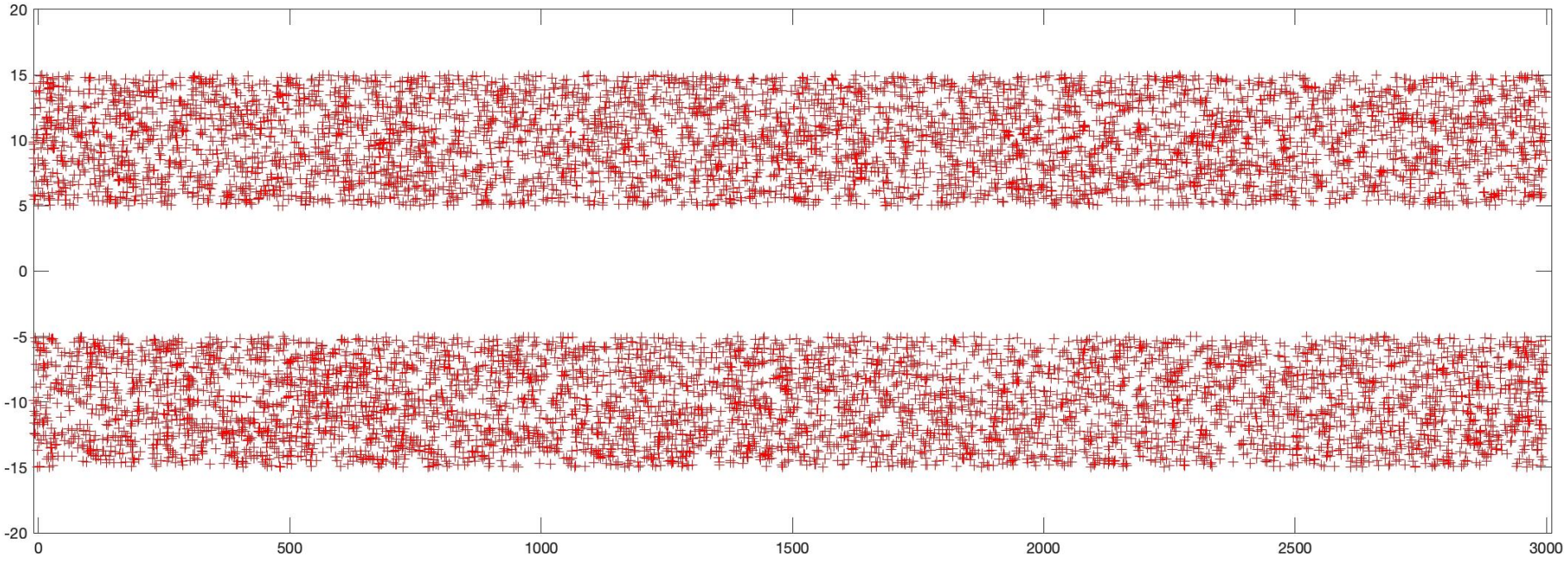
Quantity	Storage
$\hat{\mathbf{s}}_{k\|k}$	$O\left(N_k\right)$
$\mathbf{P}^{s}_{k+1\|k}$	$O\left(N_k^2\right)$

Quantity	Computation
Prediction	$O\left(N_k\right)$
Update	$O\left(N_k^2\right)$
Augment	$O\left(N_k\right)$

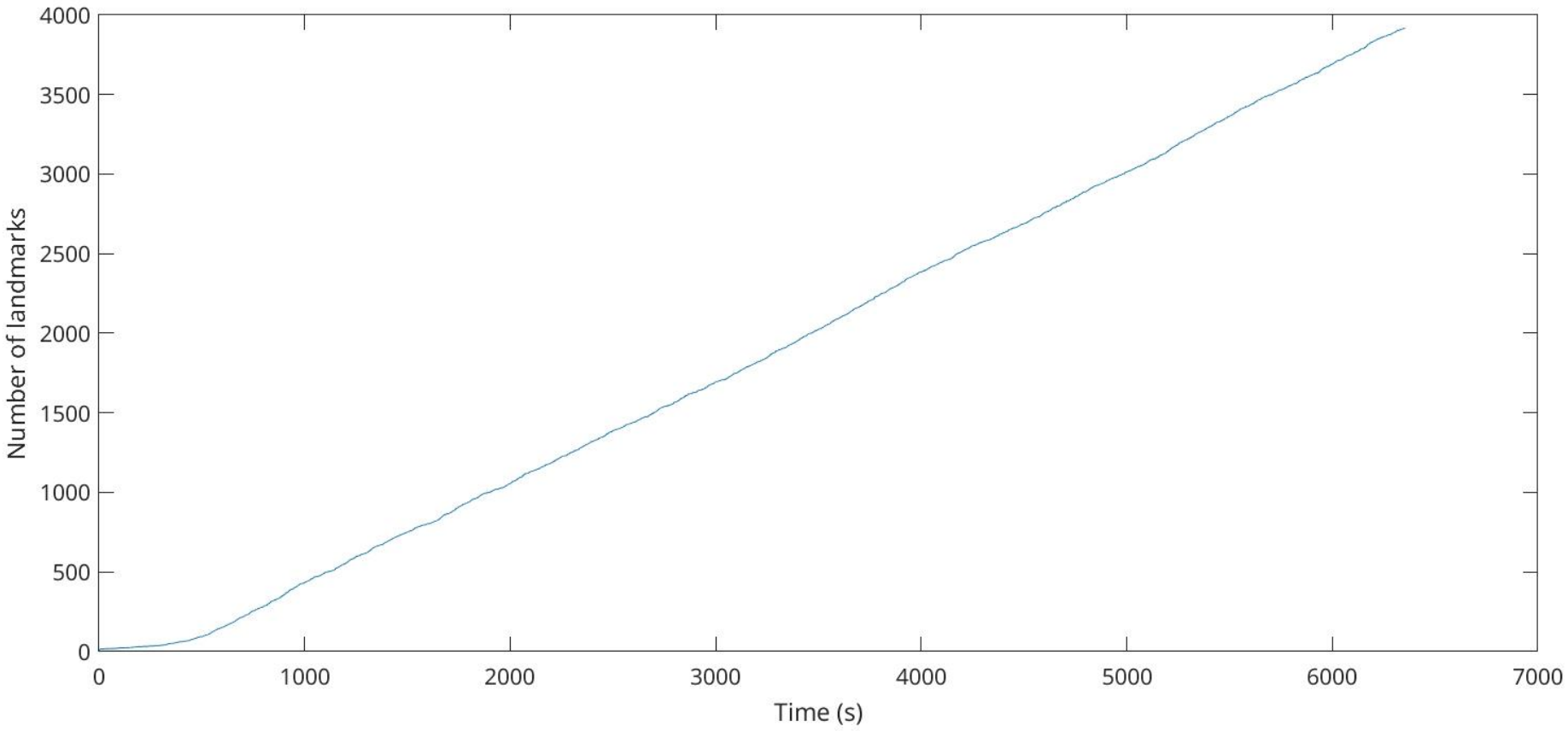
Does This Matter?

- To check scalability we set up a long raceway map full of landmarks and let the vehicle drive along it
- This means that the number of landmarks will slowly increase over time
- We can get proxies by just looking at the size of the covariance matrix and the time required per iteration

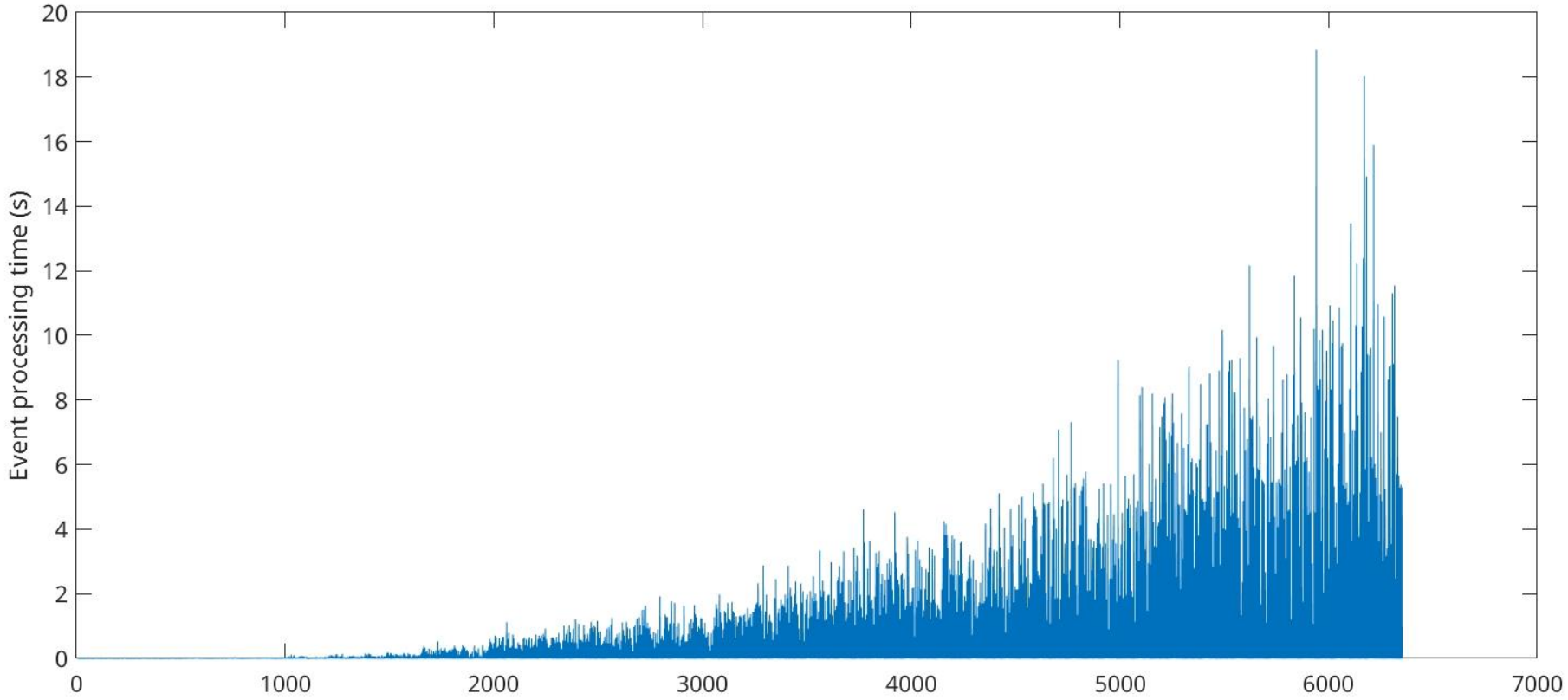
Race Way Map



Number of Landmarks



Wall Clock Time Required Per Iteration



Why Not Sparsify the Covariance?

- The reason why we have so much trouble is because of the quadratic scaling with the covariance matrix

Storage and KF-SLAM

$$\mathbf{P}^{s}_{k|k} = \begin{bmatrix} \mathbf{P}^{\mathbf{xx}}_{k|k} & \mathbf{P}^{\mathbf{x}1}_{k|k} & \mathbf{P}^{\mathbf{x}2}_{k|k} & \mathbf{P}^{\mathbf{x}3}_{k|k} & \cdots & \mathbf{P}^{\mathbf{x}N_k}_{k|k} \\ \mathbf{P}^{1\mathbf{x}}_{k|k} & \mathbf{P}^{11}_{k|k} & \mathbf{P}^{12}_{k|k} & \mathbf{P}^{13}_{k|k} & \cdots & \mathbf{P}^{1N_k}_{k|k} \\ \mathbf{P}^{2\mathbf{x}}_{k|k} & \mathbf{P}^{21}_{k|k} & \mathbf{P}^{22}_{k|k} & \mathbf{P}^{23}_{k|k} & \cdots & \mathbf{P}^{2N_k}_{k|k} \\ \mathbf{P}^{3\mathbf{x}}_{k|k} & \mathbf{P}^{31}_{k|k} & \mathbf{P}^{32}_{k|k} & \mathbf{P}^{33}_{k|k} & \cdots & \mathbf{P}^{3N_k}_{k|k} \\ \vdots & \vdots & \ddots & \vdots & & \\ \mathbf{P}^{N_k\mathbf{x}}_{k|k} & \mathbf{P}^{N_k1}_{k|k} & \mathbf{P}^{N_k2}_{k|k} & \mathbf{P}^{N_k3}_{k|k} & \cdots & \mathbf{P}^{N_kN_k}_{k|k} \end{bmatrix}$$

Why Not Sparsify the Covariance?

- The reason why we have so much trouble is because of the quadratic scaling with the covariance matrix
- Most of the terms are the cross correlations
- Do we really need all the terms in there?
- What do they do anyway?

The Critical Role of Cross Correlations

- *Long-term Convergence Behaviour*
- *Computational and Storage Costs*
- The Critical Role of Cross Correlations
- *Ways to Improve Scalability*

Sparsifying the Covariance

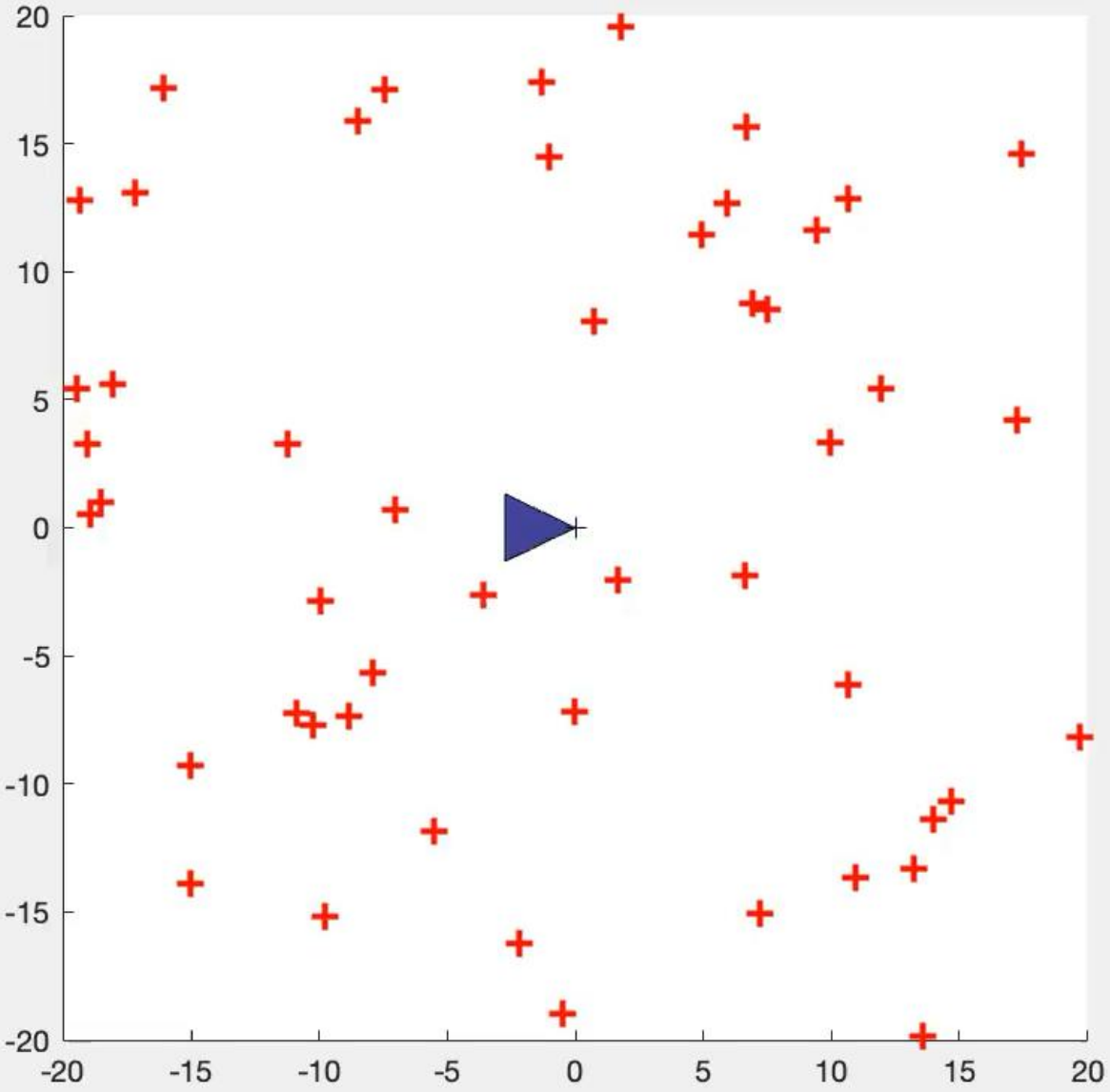
- **There are lots of ways we can do it, but the most dramatic is to force the covariance matrix to be *diagonal***

Sparsified Covariance

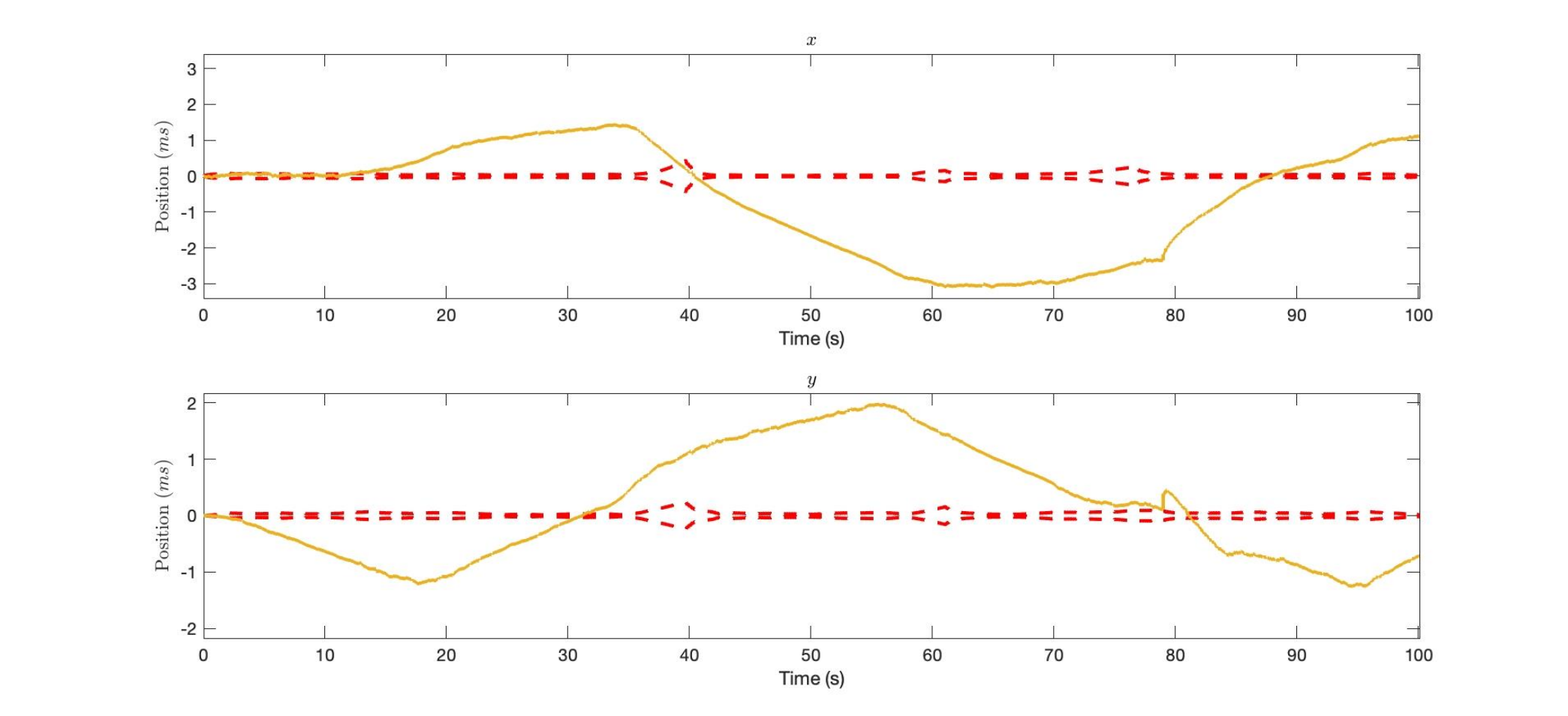
$$\mathbf{P}^{s}_{k|k} = \begin{bmatrix} \mathbf{P}^{\mathbf{xx}}_{k|k} & 0 & 0 & 0 & \cdots & 0 \\ 0 & \mathbf{P}^{11}_{k|k} & 0 & 0 & \cdots & 0 \\ 0 & 0 & \mathbf{P}^{22}_{k|k} & 0 & \cdots & 0 \\ 0 & 0 & 0 & \mathbf{P}^{33}_{k|k} & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \\ 0 & 0 & 0 & 0 & \cdots & \mathbf{P}^{N_k N_k}_{k|k} \end{bmatrix}$$

Sparsifying the Covariance

- There are lots of ways we can do it, but the most dramatic is to force the covariance matrix to be *diagonal*
- This gives a highly scalable system:
  - Storage costs are linear
  - Prediction costs are constant time
  - Update costs are constant time
  - Augmentation costs are constant time


20
15
10
5
0
-5
-10
-15
-20
-20
-15
-10
-5
0
5
10
15
20

Estimation Error



What Happened?

- The cross correlations are *not* nuisance parameters
- They explicitly tell the filter how variables link with one another
- They tell us what information is common and what information is independent
- Incorrect description of the correlation structure is the reason why Kalman filters fail; not that the distribution is non-Gaussian

So Where Next?

- So does this mean that we can’t solve the problem?
- There are a number of legitimate ways to solve the problem
- We’ll look at these next

Ways to Improve Scalability

- *Long-term Convergence Behaviour*
- *Computational and Storage Costs*
- *The Critical Role of Cross Correlations*
- **Ways to Improve Scalability**

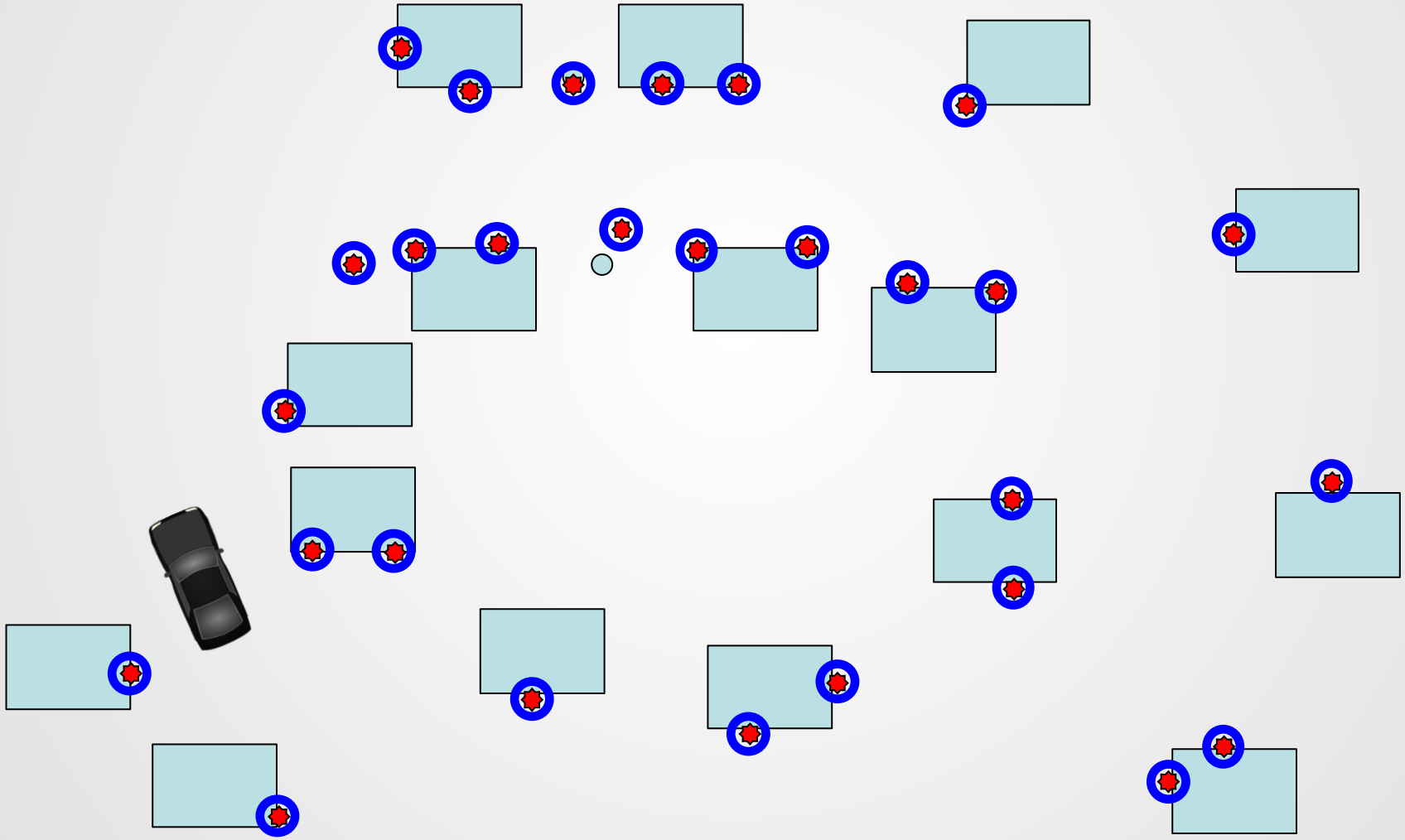
Scalability and Sub-Mapping

- There are several ways to reduce the size of the map which actually work
- Two common ones are:
  - Map pruning
  - Submapping

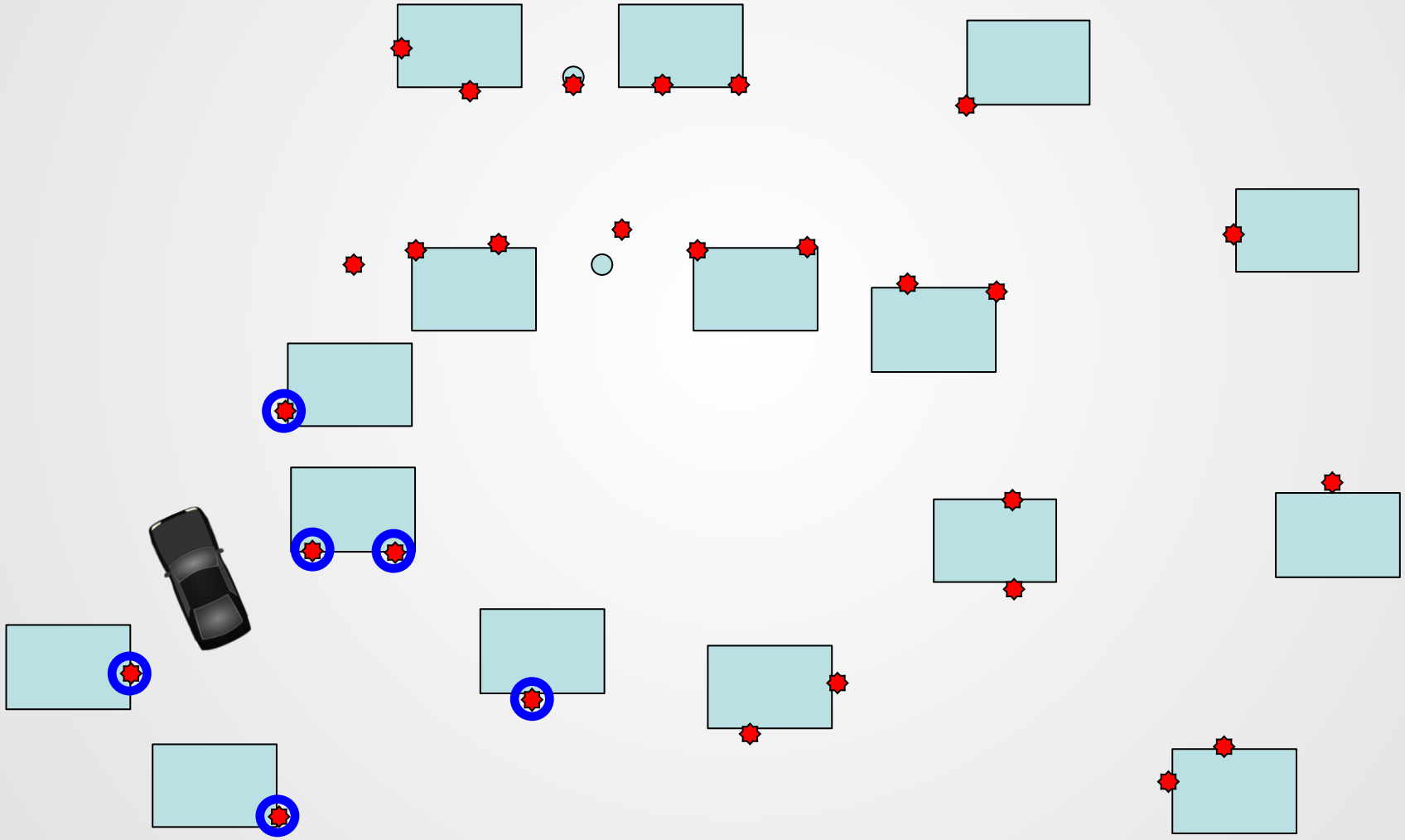
Map Pruning

- Control storage and computational costs by limiting the number of landmarks in the map
- You do this by deleting landmarks from the map
- If you encounter the landmarks again, you treat them as new

Map Pruning



Sliding Window Map Pruning (=Odometry)



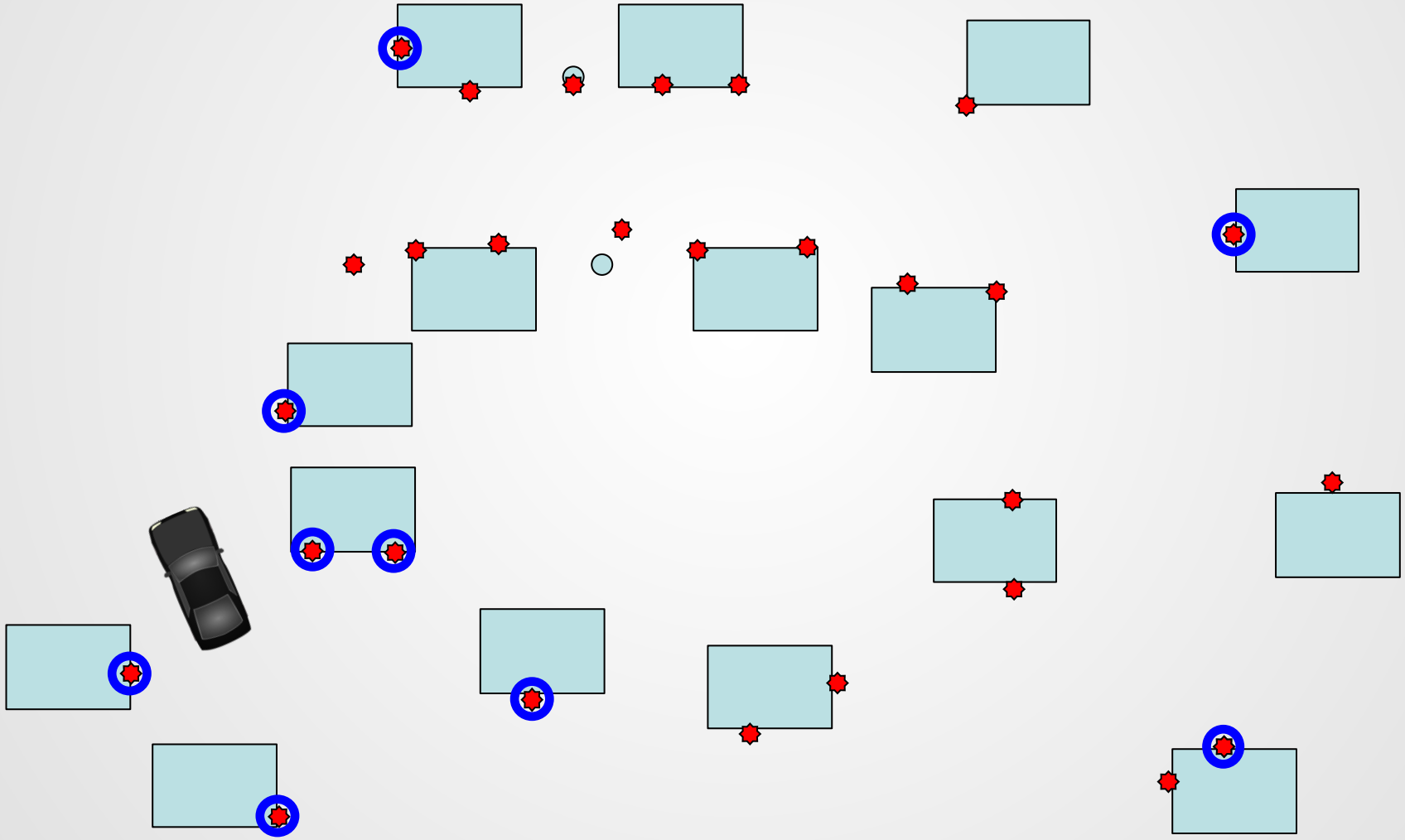
Landmark Deletion

$$\hat{\mathbf{s}}_{k|k} = \begin{pmatrix} \hat{\mathbf{x}}_{k|k} \\ \hat{\mathbf{m}}^{1}_{k|k} \\ \hat{\mathbf{m}}^{2}_{k|k} \\ \hat{\mathbf{m}}^{3}_{k|k} \\ \vdots \\ \hat{\mathbf{m}}^{N_k}_{k|k} \end{pmatrix}, \quad \mathbf{P}^{s}_{k|k} = \begin{bmatrix} \mathbf{P}^{\mathbf{xx}}_{k|k} & \mathbf{P}^{\mathbf{x}1}_{k|k} & \mathbf{P}^{\mathbf{x}2}_{k|k} & \mathbf{P}^{\mathbf{x}3}_{k|k} & \cdots & \mathbf{P}^{\mathbf{x}N_k}_{k|k} \\ \mathbf{P}^{1\mathbf{x}}_{k|k} & \mathbf{P}^{11}_{k|k} & \mathbf{P}^{12}_{k|k} & \mathbf{P}^{13}_{k|k} & \cdots & \mathbf{P}^{1N_k}_{k|k} \\ \mathbf{P}^{2\mathbf{x}}_{k|k} & \mathbf{P}^{21}_{k|k} & \mathbf{P}^{22}_{k|k} & \mathbf{P}^{23}_{k|k} & \cdots & \mathbf{P}^{2N_k}_{k|k} \\ \mathbf{P}^{3\mathbf{x}}_{k|k} & \mathbf{P}^{31}_{k|k} & \mathbf{P}^{32}_{k|k} & \mathbf{P}^{33}_{k|k} & \cdots & \mathbf{P}^{3N_k}_{k|k} \\ \vdots & \vdots & \ddots & \vdots & & \\ \mathbf{P}^{N_k\mathbf{x}}_{k|k} & \mathbf{P}^{N_k1}_{k|k} & \mathbf{P}^{N_k2}_{k|k} & \mathbf{P}^{N_k3}_{k|k} & \cdots & \mathbf{P}^{N_kN_k}_{k|k} \end{bmatrix}$$

Map Pruning

- Control storage and computational costs by limiting the number of landmarks in the map
- You do this by deleting landmarks from the map
- If you encounter the landmarks again, you treat them as new
- However, you either end up with pure odometry or a map which still grows over time
- You can try to keep important landmarks but these still grow over time

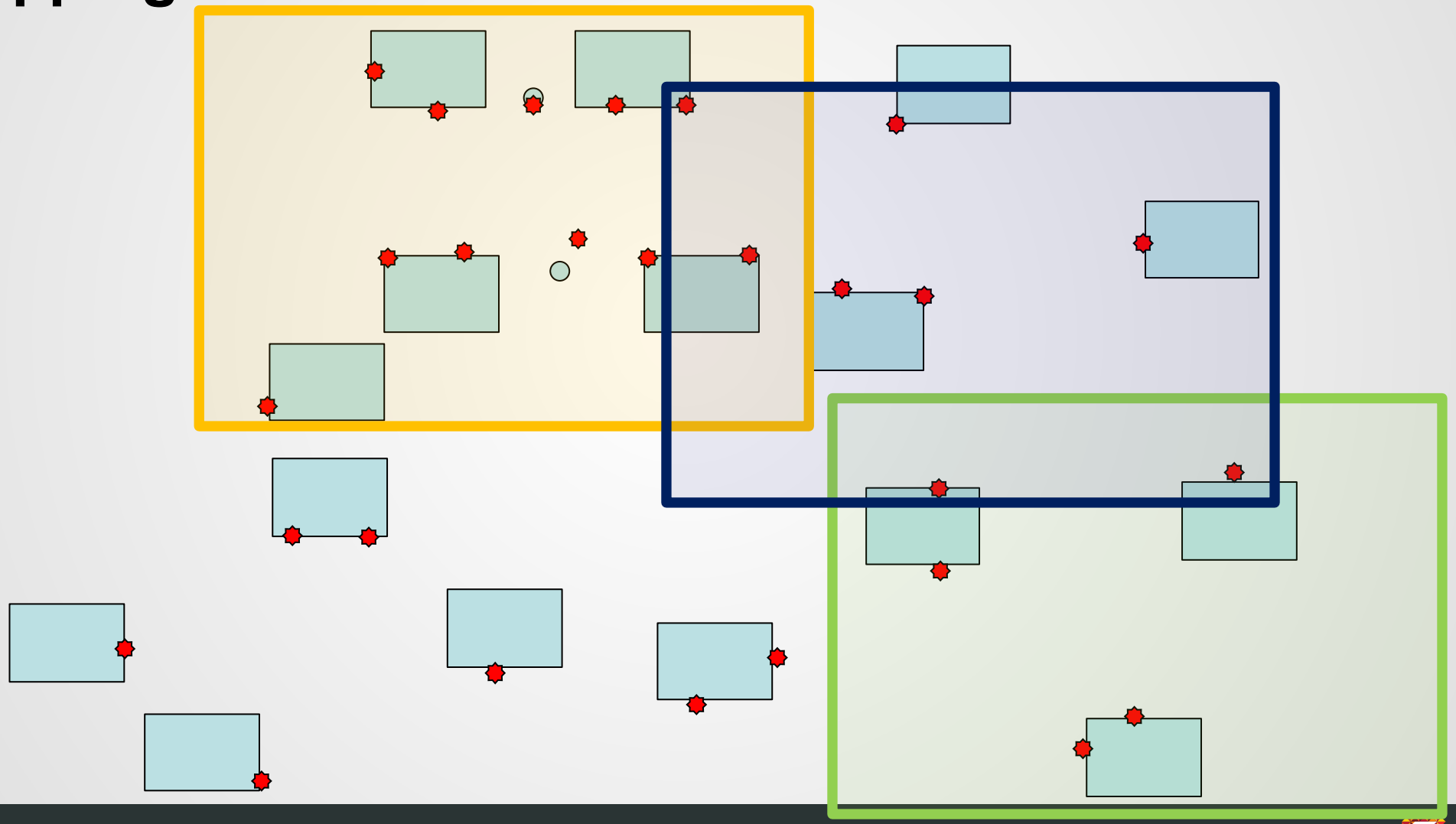
Keep Important Features Map Pruning



Sub-Mapping

- Break the environment into lots of little submaps which partially overlap one another
- For each submap run a different SLAM system
- Only a subset of SLAM systems are active at any time

Sub-Mapping



Sub-Mapping

- Break the environment into lots of little submaps which partially overlap one another
- For each submap run a different SLAM system
- Only a subset of SLAM systems are active at any time
- There's quite a lot of complexity about how to initialize the platform state between submaps
- However, this can be carefully handled to ensure problems don't arise

Summary

- LKF SLAM systems converge towards a map in which all the relative errors go to 0
- However, scaling and computational costs scale quadratically with the number of landmarks
- We can manage these problems using techniques such as submapping
- Okay… So let's see how we can now apply this to nonlinear systems